

Module 3

Q. Explain Raymond's Algorithm for Mutual Exclusion

=> refer this link (https://youtu.be/izS9J07ToRM?si=Hp_RMbs7hxxk5w6hO)

1. Raymond's algorithm is a distributed mutual exclusion algorithm used to control access to a shared resource in a distributed system.
2. Mutual exclusion means that only one process can enter the critical section at a time.
3. Raymond's algorithm is based on a token passing mechanism.
4. The algorithm organizes processes in the form of a logical tree structure.
5. Only the process that has the token is allowed to enter the critical section.

Basic Concepts

1. Each process in the system is represented as a node in a logical tree.
2. One node initially holds the token which gives permission to enter the critical section.
3. Each node maintains a pointer called a holder which points to the neighbor node that is closer to the token.
4. Each node also maintains a request queue to store requests from other nodes.
5. When a process wants to enter the critical section, it must obtain the token.

Working of Raymond's Algorithm (Step by Step)

1. When a process wants to enter the critical section, it sends a request message to its holder node.
2. The request message travels along the tree towards the node that currently holds the token.
3. Each intermediate node adds the request to its local request queue.
4. If a node does not have the token, it forwards the request to its holder.
5. When the token-holding node receives the request, it sends the token to the requesting node.
6. The token travels along the same path in reverse direction to reach the requester.
7. When the requesting node receives the token, it enters the critical section.
8. After completing execution, the node checks its request queue.
9. If there are pending requests, the token is passed to the next node in the queue.
10. If there are no pending requests, the node keeps the token until a new request arrives.

Example

1. Suppose there are five processes arranged in a tree structure.
2. Process P1 initially holds the token.
3. Process P5 requests access to the critical section.
4. The request moves from P5 to its holder and finally reaches P1.
5. P1 sends the token back to P5 through the same path.
6. P5 enters the critical section and executes its task.

Q. Explain Bully Election Algorithm with Suitable Example

=> refer this link (<https://youtu.be/PMjWOWDp9Vo?si=NxGy6UuW0N2ZjhGR>)

1. Bully algorithm is a leader election algorithm used in distributed systems to select a coordinator process.
2. A coordinator is responsible for managing tasks and controlling system activities.
3. The bully algorithm is used when the current coordinator fails or stops working.
4. The algorithm selects the process with the highest priority number as the new coordinator.
5. Each process in the system has a unique ID, and higher ID means higher priority.
6. The algorithm is called bully because the highest priority process forces itself to become the leader.

Assumptions of Bully Algorithm

1. Each process has a unique identification number.
2. All processes know the IDs of other processes in the system.
3. The system can detect process failure using timeout mechanisms.
4. Communication between processes is reliable.
5. The process with the highest ID should become the coordinator.

Working of Bully Algorithm (Step by Step)

1. When a process detects that the coordinator is not responding, it starts an election.
2. The process sends an election message to all processes with higher IDs than itself.
3. If no higher-ID process responds, the process becomes the coordinator.
4. The process then sends a coordinator message to all other processes announcing itself as leader.
5. If a higher-ID process responds with an OK message, the lower-ID process stops the election.
6. The higher-ID process then starts a new election by sending messages to processes with even higher IDs.
7. This process continues until the highest-ID active process becomes the coordinator.

Example of Bully Algorithm

1. Suppose there are five processes with IDs P1, P2, P3, P4, and P5.
2. Process P5 has the highest ID and is the coordinator.
3. If P5 fails, process P2 detects the failure and starts an election.
4. P2 sends election messages to P3, P4, and P5.
5. P3 and P4 respond with OK messages because they have higher IDs.
6. P2 stops its election process.
7. P4 then starts another election by sending messages to P5.
8. Since P5 is down, P4 receives no response.
9. P4 becomes the new coordinator and sends a coordinator message to all processes.

Q. What is Logical Clock? Why are Logical Clocks Required in Distributed Systems? How Lamport Synchronizes Logical Clock? Which Events are Said to be Concurrent in Lamport's Timestamp?

=> refer these links (<https://youtu.be/nps88TsKFE4?si=CenN-OfkFOR2YOe7> ,
<https://youtu.be/taRbDdlAK7s?si=cSfiZdHkfmbrupWv>)

Logical Clock

1. A logical clock is a mechanism used in distributed systems to order events without using physical time.
2. Logical clocks assign numbers (timestamps) to events to show their order of occurrence.
3. Logical clocks help determine which event happened before another event in a distributed environment.
4. Logical clocks do not represent actual time but represent the sequence of events.
5. Logical clocks are necessary because each computer in a distributed system has its own physical clock which may not be synchronized.
6. An example is assigning sequence numbers to messages to track their order.

Why Logical Clocks are Required in Distributed Systems

1. Distributed systems do not have a global clock shared by all processes.
2. Physical clocks of different machines may differ due to clock drift.
3. It is difficult to determine the exact order of events using physical time.
4. Logical clocks help maintain correct event ordering across multiple processes.
5. Logical clocks are useful for synchronization, coordination, and consistency.
6. Logical clocks help detect causality relationships between events.
7. Logical clocks are important for algorithms such as mutual exclusion and distributed databases.
8. An example is ordering transactions in distributed databases.

Lamport Logical Clock Synchronization (Lamport Timestamp Algorithm)

Lamport proposed a method to synchronize logical clocks using simple rules.

Rules for Lamport Logical Clock

1. Each process maintains a local logical clock counter.
2. Initially, the logical clock value of each process is set to zero.

Rule 1: Internal Event

1. Before executing any event within a process, the process increments its logical clock by 1.
2. The event is assigned the updated clock value as its timestamp.

Rule 2: Sending a Message

1. When a process sends a message, it increments its clock by 1.
2. The message carries the timestamp of the sending process.

Rule 3: Receiving a Message

1. When a process receives a message, it compares its clock value with the received timestamp.
2. The process updates its clock to the maximum of its own clock and the received timestamp.
3. The process then increments its clock by 1.
4. This updated value becomes the timestamp of the receiving event.

Result

1. Lamport clocks ensure that if one event happens before another event, then the timestamp of the first event is smaller.
2. This property is called the happened-before relationship.

Example: If Process P1 sends a message at time 5 and Process P2 receives it, P2 updates its clock to a value greater than 5.

Concurrent Events in Lamport Timestamp

1. Two events are said to be concurrent if there is no causal relationship between them.
2. Concurrent events do not affect each other directly or indirectly.
3. If event A does not happen before event B and event B does not happen before event A, then they are concurrent.
4. Concurrent events may have timestamps that cannot determine exact ordering using causality.
5. Concurrent events typically occur in different processes without communication between them.

Example: Two users editing different documents at the same time on separate computers without interaction.

Q. Explain Ricart-Agrawala's Algorithm and How it Optimizes the Message Overhead in Achieving Mutual Exclusion

=> refer this link (<https://youtu.be/MUmCrwyzMCE?si=iCjgUrE4xdF2GcHH>)

Introduction

1. Ricart-Agrawala algorithm is a distributed mutual exclusion algorithm used to control access to a shared resource.
2. Mutual exclusion ensures that only one process enters the critical section at a time.
3. The algorithm is based on timestamp ordering using Lamport logical clocks.
4. The algorithm does not use any central coordinator or token.
5. Each process communicates directly with other processes to request permission.
6. The algorithm improves efficiency compared to earlier algorithms by reducing message overhead.

Basic Idea of Ricart-Agrawala Algorithm

1. A process must obtain permission from all other processes before entering the critical section.
2. Permission is granted using reply messages.
3. Timestamps are used to decide which process gets priority.
4. The process with the smallest timestamp gets access to the critical section first.
5. If two timestamps are equal, the process ID is used to break the tie.

Working of Ricart-Agrawala Algorithm (Step by Step)

1. When a process wants to enter the critical section, it increments its logical clock.
2. The process sends a REQUEST message with its timestamp to all other processes.
3. When another process receives the request, it checks its own state.

There are three possible cases:

4. If the receiving process is not interested in the critical section, it immediately sends a REPLY message.
5. If the receiving process is already inside the critical section, it delays the reply until it exits.
6. If the receiving process also wants to enter the critical section, it compares timestamps.
7. The process with the smaller timestamp gets priority.
8. The process with the larger timestamp delays its reply.
9. The requesting process waits until it receives REPLY messages from all other processes.
10. After receiving all replies, the process enters the critical section.
11. After finishing execution, the process sends delayed replies to other waiting processes.

Example

1. Suppose there are three processes P1, P2, and P3.
2. P1 and P2 request the critical section at the same time.
3. P1 has timestamp 5 and P2 has timestamp 8.
4. P1 gets priority because its timestamp is smaller.
5. P2 waits until P1 completes execution.
6. After P1 exits, P2 receives the remaining replies and enters the critical section.

How Ricart-Agrawala Optimizes Message Overhead

1. Earlier algorithms like Lamport's mutual exclusion algorithm required three types of messages: REQUEST, REPLY, and RELEASE.
2. Ricart-Agrawala algorithm removes the need for RELEASE messages.
3. The algorithm uses only REQUEST and REPLY messages.
4. The total number of messages required is reduced to $2(N-1)$ messages for N processes.
5. Reducing message types decreases communication overhead and improves efficiency.
6. The algorithm avoids unnecessary broadcast messages after leaving the critical section.
7. The delayed reply mechanism replaces the need for release notification.

Q. Explain Chandy–Misra–Haas Algorithm for Distributed Deadlock Detection

=>

Introduction

1. Chandy–Misra–Haas algorithm is a distributed algorithm used to detect deadlocks in distributed systems.
2. A deadlock occurs when a set of processes are waiting for each other indefinitely and none of them can proceed.
3. The algorithm is based on the concept of a wait-for graph.
4. The algorithm detects cycles in the wait-for graph to identify deadlocks.
5. The algorithm works without a central controller and uses message passing between processes.
6. The algorithm is efficient because it sends messages only when deadlock is suspected.

Basic Concept

1. In distributed systems, processes may run on different machines and wait for resources held by other processes.
2. A wait-for relationship exists when one process is waiting for another process to release a resource.
3. Deadlock occurs when there is a circular wait among processes.
4. The algorithm uses special messages called probe messages to detect cycles.
5. If a probe message returns to the original process, a deadlock is detected.

Types of Chandy–Misra–Haas Algorithm

1. The algorithm has different versions for different resource models.
2. The most common version is the AND model where a process waits for multiple resources simultaneously.
3. Another version is the OR model where a process waits for any one of multiple resources.

Working of Chandy–Misra–Haas Algorithm (Step by Step)

1. When a process becomes blocked while waiting for a resource, it initiates deadlock detection.
2. The blocked process sends a probe message to the process holding the required resource.
3. The probe message contains three values: initiator process ID, sender process ID, and receiver process ID.
4. The process receiving the probe checks whether it is also waiting for another process.
5. If the receiving process is not blocked, it discards the probe message.
6. If the receiving process is blocked, it forwards the probe message to the process it is waiting for.
7. The probe message travels along the wait-for chain.
8. If the probe message returns to the initiator process, a cycle exists.

9. The cycle confirms that a deadlock has occurred.
10. After detection, recovery mechanisms such as terminating processes or releasing resources may be applied.

Example

1. Suppose three processes P1, P2, and P3 are involved in resource waiting.
2. Process P1 is waiting for P2, process P2 is waiting for P3, and process P3 is waiting for P1.
3. P1 starts the detection by sending a probe message to P2.
4. P2 forwards the probe to P3 because it is also waiting.
5. P3 forwards the probe to P1 because it is waiting for P1.
6. The probe returns to P1, indicating a deadlock cycle.

Advantages

1. The algorithm is fully distributed and does not require a central coordinator.
2. The algorithm reduces unnecessary message passing by detecting deadlocks only when needed.
3. The algorithm works efficiently in large distributed systems.

Disadvantages

1. The algorithm may generate extra messages in complex systems with many processes.
2. The algorithm requires reliable communication between processes.
3. Deadlock detection may take time depending on system size.